Module 2
String Processing Mechanics

Agenda

1. Introduction to Strings
2. Strings as Immutable Sequences
3. Indexing Mechanics
4. Slicing
5. String Operations
6. String Methods
7. String Formatting / Interpolation

© Copyright Intellipaat. All rights reserved.

Introduction to Strings

© Copyright Intellipaat. All rights reserved.

Introduction to Strings

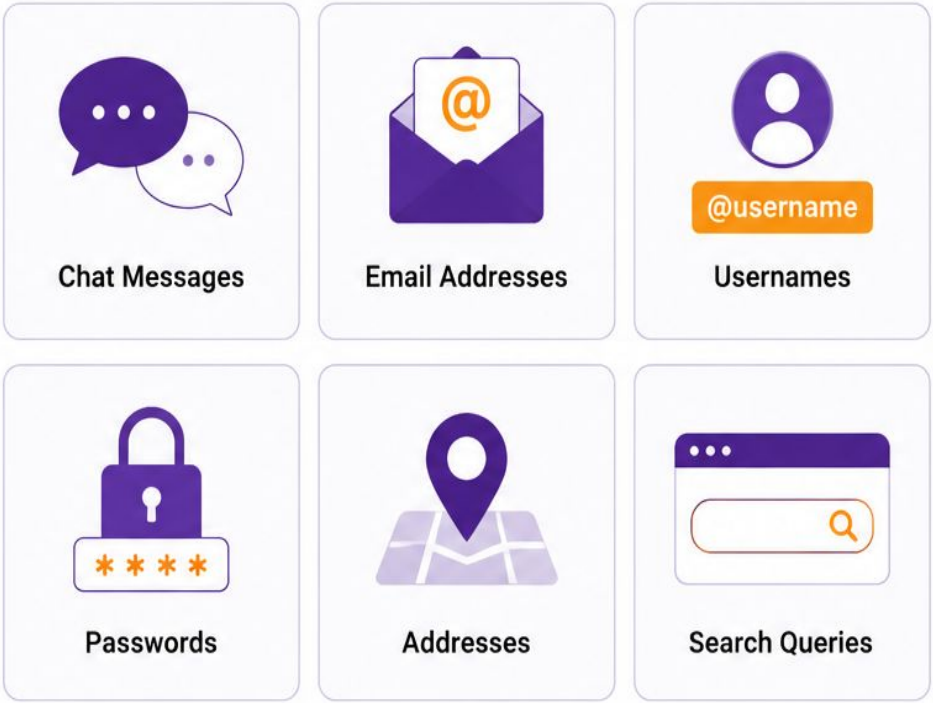
Strings are used in programming to work with **text**.

- Store and process textual data
- Appear in almost every application
- Essential for **user interaction**, **communication**, and **information display**

Key Takeaway

Strings help programs understand, manipulate, and display text information.

Where do we see strings in real life?



© Copyright Intellipaat. All rights reserved.

What is a string?

A string is a **sequence of characters** enclosed within **quotes**.

Examples:

`"Hello"`	→ A word
`'Python'`	→ A programming language
`"12345"`	→ A number (still a string)
`'manan@gmail.com'`	→ An email address

Key Point

Anything inside quotes in Python is treated as a string.

Strings can contain:

- **Letters** (A–Z, a–z)
- **Numbers** (0–9)
- **Symbols** (!, @, #, $, ...)
- **Spaces** (blank space between words)

© Copyright Intellipaat. All rights reserved.

Why Strings Matter

Strings are the **backbone of modern** software and everyday **digital interactions**.

Enable Communication
Allow users and systems to share messages and information.

Store Important Data
Hold names, IDs, emails, addresses, and other textual information.

Power User Interfaces
Display labels, menus, prompts, and notifications.

Make Data Meaningful
Turn raw text into insights, search results, and decisions.

Key Takeaway

Without strings, computers cannot understand, display, or process the text we rely on every day.

Imagine an app without strings...

WITHOUT STRINGS		WITH STRINGS	
Login	_____	Login	manan_123
Chat	_____	Chat	Hello! How are you?
Search	_____	Search	python tutorial
Email	_____	Email	manan@gmail.com
File Name	_____	File Name	report_final.txt
Nothing works. Everything is blank.		Everything works. Text makes it possible!	

VS

© Copyright Intellipaat. All rights reserved.

Creating Strings in Python

In Python, strings can be created using different types of **quotes**.

- **Single quotes:** ‘ ’
- **Double quotes:** “ ”
- **Triple quotes:** ‘’’ ’’’ or “”” “””

Key Point

No matter which quotes you use, Python treats the content as a string.

Single Quotes

Use single quotes to create strings.

```
name = 'Manan'
print(name)
```

↓

Manan

Double Quotes

Use double quotes to create strings.

```
city = "Pune"
print(city)
```

↓

Pune

Triple Quotes

Use triple quotes for multi-line strings.

```
text = """Welcome
    to Python!"""
print(text)
```

↓

Welcome
to Python!

© Copyright Intellipaat. All rights reserved.

Types of String Content

Strings in Python can hold different kinds of **text** and **characters**.

Examples

`"A"`	→ A single character
`"Hello World"`	→ A sentence or phrase
`"12345"`	→ Numbers (**still a string**)
`"@intellipaat"`	→ Special symbols
`"Room No 101"`	→ Alphanumeric text with spaces

Key Takeaway

Anything inside quotes in Python is treated as a string, no matter what it contains.

Class Activity

Which of the following are **strings**?

`100` ✗ **Not a string** (It's an integer)	`"100"` ✓ **String** (Number inside quotes)	`3.14` ✗ **Not a string** (It's a float)	`"3.14"` ✓ **String** (Float inside quotes)
`True` ✗ **Not a string** (It's a boolean)	`"True"` ✓ **String** (Boolean inside quotes)	`" "` ✓ **String** (Space is also a character)	`"@123"` ✓ **String** (Symbols + numbers)

© Copyright Intellipaat. All rights reserved.

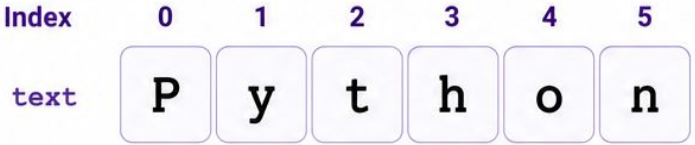
What Happens Inside a String?

A string is a sequence of characters stored in a specific order.

Example

```
text = "Python"
```

Visual Representation



Key Takeaway

Strings store characters in order, allowing us to access each character using its index.

Think About It

- How does Python access individual characters?
- Can we change any character in a string?
- What happens if we try to modify a string?

Next Up:
Strings as Immutable Sequences

© Copyright Intellipaat. All rights reserved.

Hands-On: Introduction to Strings

© Copyright Intellipaat. All rights reserved.

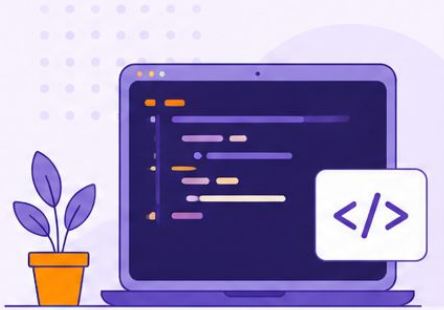
Hands-On: Introduction to Strings

Problem Statement

Write a Python program to create and print:

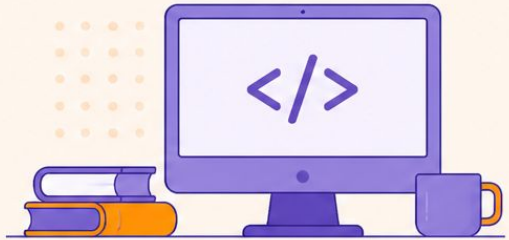
- A course name using double quotes: **Python Programming**
- A level using single quotes: **Beginner**
- A multi-line message using triple quotes:

  Learn
  Practice
  Grow



Expected Output

```
Course: Python Programming
Level: Beginner
Message:
Learn
Practice
Grow
```



© Copyright Intellipaat. All rights reserved.

Strings as Immutable Sequences

© Copyright Intellipaat. All rights reserved.

Strings are Sequences

In Python, a string is a **sequence**, which means:

1. **Data stored in order**
   Elements are arranged in a specific order.

2. **Position matters**
   Changing the position changes the meaning.

3. **Each item can be accessed individually**
   We can access each character using its index.

Example

```
text = "Hello"
```

Visual Representation

Index	0	1	2	3	4
text	H	e	l	l	o

Sequential Order

H → e → l → l → o

Other Sequences in Python

- **Strings**
  Sequence of characters enclosed in quotes.
- **Lists**
  Ordered collection of items, mutable.
- **Tuples**
  Ordered collection of items, immutable.

Key Takeaway

Strings are sequences of characters. The order and position of each character matter, and we can access them one by one using their index.

© Copyright Intellipaat. All rights reserved.

Immutable Means Unchangeable

Once a string is created, its characters **cannot be modified.**

Example

```
name = "Python"
```

Visual Representation

Index	0	1	2	3	4	5
name	P	y	t	h	o	n

Cannot Change Directly

Trying to change: P → J

This is **not allowed** in Python!

Real-World Analogy



Think of a printed book.

- Once it is printed, you cannot change any letter on a page.
- You can't erase or overwrite a character directly.

Instead of changing, you create a new edition (a **new string**) with the desired changes.

Key Takeaway

Strings are immutable, meaning their characters cannot be changed once the string is created.

© Copyright Intellipaat. All rights reserved.

Can We Modify a String?

Let's try to **change a character** in a string.

Example

```
word = "Hello"
word[0] = "Y"
```

What happens?

Python throws an error:

```
TypeError:
'str' object does not
support item assignment
```

Why?

Strings are **immutable**.

You cannot modify individual characters in a string.

Important!

Python does not allow direct modification of string characters.

Key Takeaway

You cannot change characters in a string.
Python will raise a **TypeError**.

© Copyright Intellipaat. All rights reserved.

Modification Creates a New String

We cannot change a string directly.
Instead, we create a **new string** with the desired changes.

Example

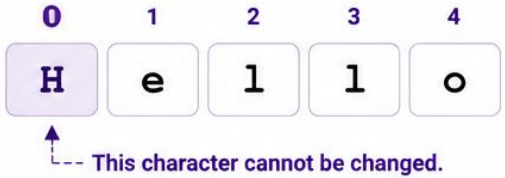
```
word = "Hello"
new_word = "Y" + word[1:]
```

Output

```
new_word
'Yello'
```

What's Happening?

Before (Original String)



After (New String Created)



Key Takeaway

- Strings are immutable.
- Any "modification" results in the creation of a new string.
- The original string remains unchanged.

Remember

Immutability ensures that strings are safe from accidental changes.

© Copyright Intellipaat. All rights reserved.

Hands-On: Strings as Immutable Sequences

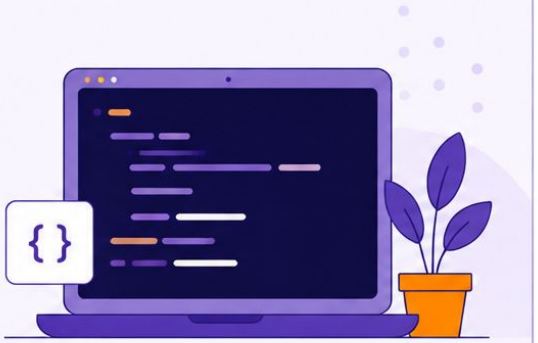
© Copyright Intellipaat. All rights reserved.

Hands-On: Strings as Immutable Sequences

Problem Statement

Given `word = "Coding"`.

- Create a new string `"Loading"` by replacing the first character of `word` with `'L'`.
- Do **not** modify the original string.
- Print both the original string and the new string.



Expected Output

```
Original String: Coding
New String: Loading
```



© Copyright Intellipaat. All rights reserved.

Indexing Mechanics

© Copyright Intellipaat. All rights reserved.

Indexing

Every character in a string has a position called an **index**.



Every Character Has a Position

Characters in a string are arranged in a specific order.



Positions Are Called Indexes

Indexes help Python identify and access characters.



Python Uses Indexing

Indexing (using square brackets []) lets us access characters easily.



Key Takeaway

Indexes are numbers that represent the position of characters in a string.
Python uses indexing to access individual characters.

© Copyright Intellipaat. All rights reserved.

Positive Indexing

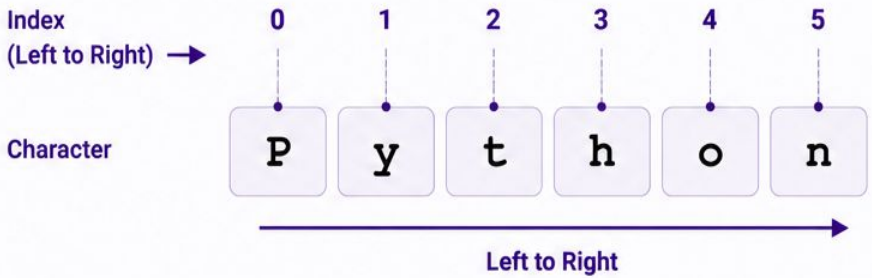
In Python, indexing starts from **left to right** with **0**.

Example

```
word = "Python"
```

Each character has an index starting from **0**.

Indexing Visualization



Accessing Characters

```
word = "Python"
print(word[0])   # Output: P
print(word[1])   # Output: y
print(word[2])   # Output: t
```

Output

```
P
y
t
```

Note

The first character is at index **0**, the second at index **1**, and so on.

Key Takeaway

Positive indexing starts from **0** on the left side and moves towards the right.

© Copyright Intellipaat. All rights reserved.

Negative Indexing

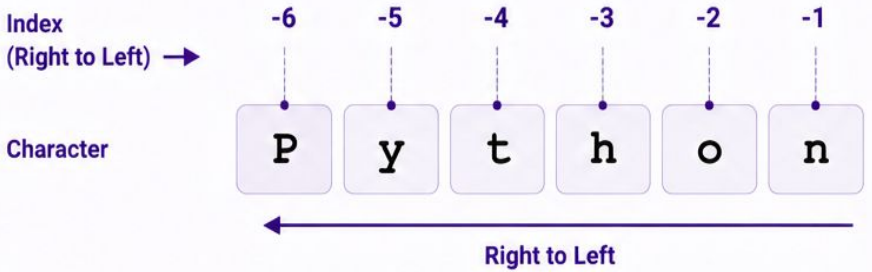
In Python, negative indexing starts from **right to left** with **-1**.

Example

```
word = "Python"
```

The last character has index **-1**,
the second last has index **-2**, and so on.

Indexing Visualization



Accessing Characters

```
word = "Python"

print(word[-1])  # Output: n
print(word[-2])  # Output: o
print(word[-6])  # Output: P
```

Output

```
n
o
P
```

Note

Negative indexing is very useful
when working with the end of a string.

Key Takeaway

Negative indexing starts from **-1** at the right end and moves towards the left.

© Copyright Intellipaat. All rights reserved.

Accessing Characters Using Indexes

We can access individual characters in a string using their index.

EXAMPLE

```
word = "Python"
print(word[0])
print(word[3])
```

OUTPUT

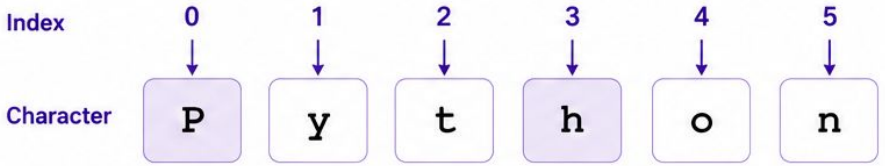
```
P
h
```

EXPLANATION

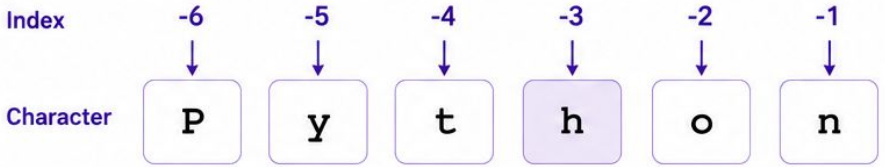
- `word[0]` accesses the character at index 0 → 'P'
- `word[3]` accesses the character at index 3 → 'h'

VISUAL REPRESENTATION

Positive Indexing (Left to Right)



Negative Indexing (Right to Left)



KEY TAKEAWAY

Use the index (positive or negative) inside square brackets `[]` to access the character at that position in a string.

© Copyright Intellipaat. All rights reserved.

What Happens with an Invalid Index?

If we try to access an index that is out of range, Python raises an **IndexError**.

Example

```
word = "Python"
print(word[10])  # Invalid index
```

Output

```
IndexError: string index out of range
```

Explanation

The string "Python" has valid indexes from -6 to 5 (or 0 to 5).
Index 10 is out of this range, so Python raises an **IndexError**.

Valid Index Range

```
word = "Python"
```

Positive Indexes (Left to Right)

Index	0	1	2	3	4	5
Character	P	y	t	h	o	n

Negative Indexes (Right to Left)

Index	-6	-5	-4	-3	-2	-1
Character	P	y	t	h	o	n

Any index < -6 or > 5 is invalid for this string.

Key Takeaway

Always make sure the index is within the valid range.
Otherwise, Python will raise an **IndexError**.

© Copyright Intellipaat. All rights reserved.

Hands-On: String Indexing with Positive and Negative Indexes

© Copyright Intellipaat. All rights reserved.

Hands-On: String Indexing with Positive and Negative Indexes

Problem Statement

Given word = "Programming",

use positive and negative indexes to display:

- The second character
- The fifth character
- The last character



Expected Output

```
Second Character: r
Fifth Character:  r
Last Character:   g
```



© Copyright Intellipaat. All rights reserved.

Slicing

© Copyright Intellipaat. All rights reserved.

String Slicing

Slicing allows us to access a part (**substring**) of a string by specifying a **start** and **end** index.



Extract Substrings

Get a portion of the string using indexes.



Slicing Operator

Use the colon (:) inside square brackets [] to slice.



Flexible and Powerful

Supports start, end, and step values for advanced slicing.



Very Useful

Helps in data extraction, manipulation, and more.



Key Takeaway

Slicing lets us access multiple characters from a string efficiently.
Remember: the start index is **included**, but the end index is **excluded**.

© Copyright Intellipaat. All rights reserved.

Slicing Syntax

We use the **colon (:)** inside square brackets **[ ]** to slice a string.

Syntax

```
string[start:end]
```

- `start` → starting index (**included**)
- `end` → ending index (**excluded**)
- Extracts characters from **start** up to, but not including, **end**.

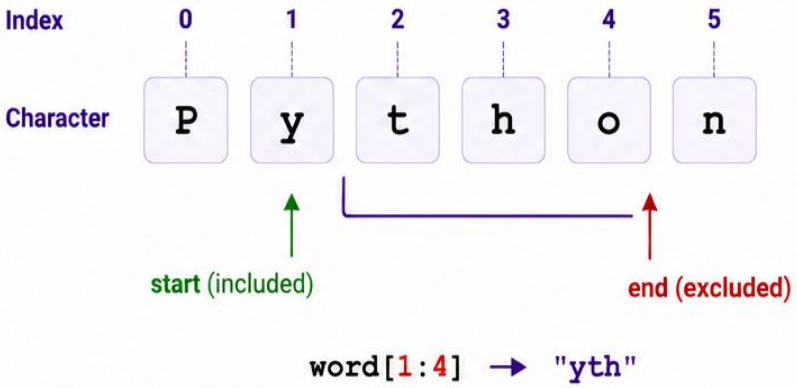
Key Rule

Start index is **included**,
End index is **excluded**.

Visual Explanation

Example:

```
word = "Python"
```




```
word[1:4] → "yth"
```

Key Takeaway

Slicing helps us extract a portion of a string using **start** and **end** indexes.

© Copyright Intellipaat. All rights reserved.

Slicing Example

Let's slice a part of a string using **start** and **end** indexes.

EXAMPLE

```
text = "LearnPython"
print(text[2:8])
```

OUTPUT

```
arnPyt
```

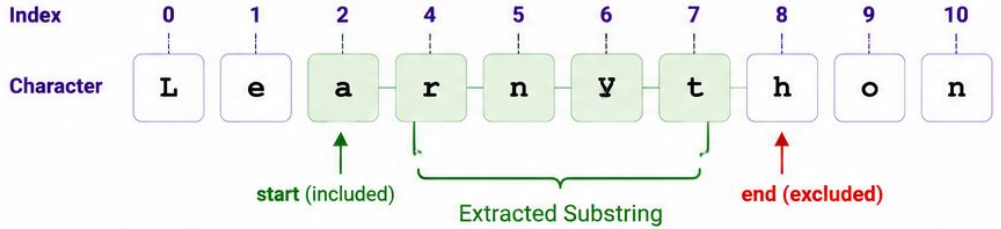
EXPLANATION

- Start index 2 is **included** → character at index 2 is 'a'.
- End index 8 is **excluded** → characters up to index 7 are taken.
- So, characters at indexes 2, 3, 4, 5, 6, 7 are extracted.

VISUAL REPRESENTATION

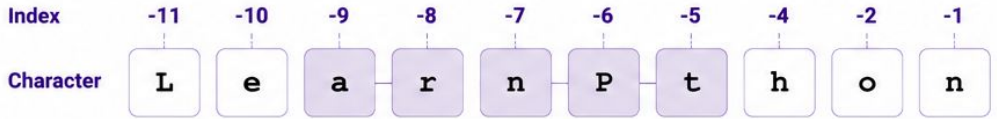
```
text = "LearnPython"
```

Positive Indexing (Left to Right)




```
text[2:8]  →  "arnPyt"
```

Negative Indexing (Right to Left)



KEY TAKEAWAY

In slicing, the start index is **included**, but the end index is **excluded**.

© Copyright Intellipaat. All rights reserved.

Slicing with Omitted Start or End

We can omit the **start** index or **end** index in slicing.

Example

```
word = "Python"
print(word[:3])   # Omit start
print(word[2:])   # Omit end
```

Output

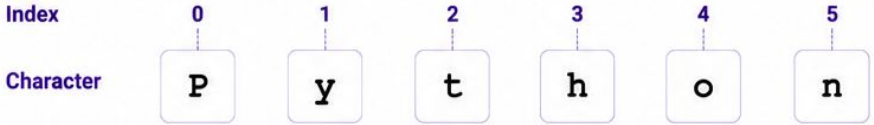
```
Pyt
thon
```

Explanation

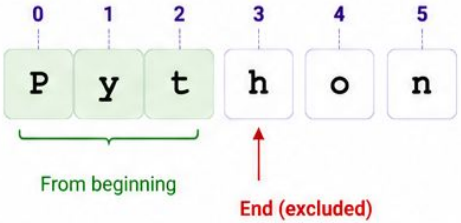
- `word[:3]` → starts from the beginning up to index 3 (excluded).
- `word[2:]` → starts from index 2 up to the end of the string.

Visual Representation

`word = "Python"`

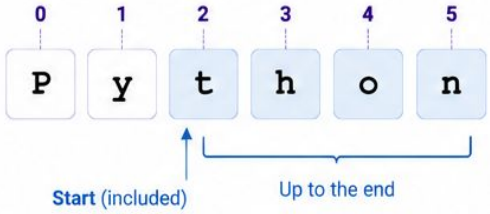


1) Omitting Start Index



`word[:3]` → `"Pyt"`

2) Omitting End Index



`word[2:]` → `"thon"`

Key Takeaway

If you omit **start**, slicing begins from the start of the string.
If you omit **end**, slicing goes until the end of the string.

© Copyright Intellipaat. All rights reserved.

Slicing with Step

We can add a **step** value to skip characters while slicing.

Example

```
word = "Python"
print(word[0:6:2])   # step = 2
```

Output

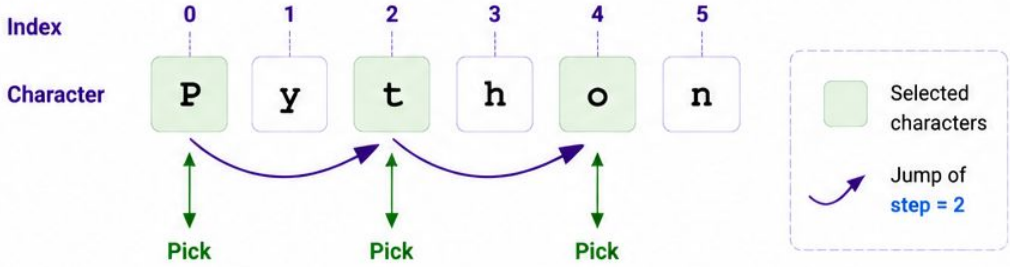
```
Pto
```

Explanation

- **start** = 0 → begin from index 0 (included)
- **end** = 6 → go up to index 6 (**excluded**)
- **step** = 2 → pick every 2nd character
- Characters taken: index 0, 2, 4
  → `'P'`, `'t'`, `'o'`

Visual Representation

```
word = "Python"
```




```
word[0:6:2]  →  "Pto"
```

If **step** is positive, slicing goes left to right.
If **step** is **negative**, slicing goes right to left (we'll see this next!).

Key Takeaway

The **step** value lets us skip characters. **step = 2** means pick every 2nd character.

© Copyright Intellipaat. All rights reserved.

Reversing a String with Slicing

We can reverse a string easily using slicing with a **negative** step.

Example

```
word = "Python"
print(word[::-1])  # Reverse string
```

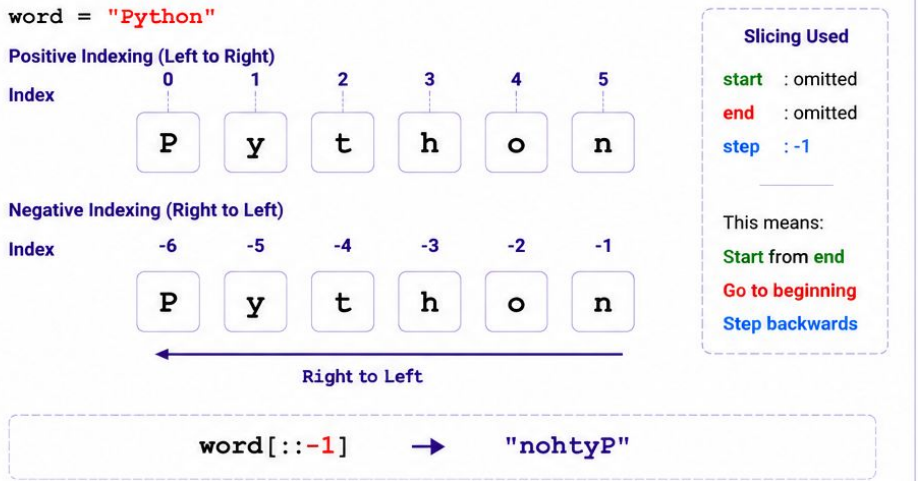
Output

```
nohtyP
```

Explanation

- **start** is omitted → begin from the end
- **end** is omitted → go till the beginning
- **step = -1** → move backwards (right to left)
- All characters are taken in reverse order.

Visual Representation



Key Takeaway

Using a step of **-1** in slicing **reverses** the string.
It's a powerful and easy trick!

© Copyright Intellipaat. All rights reserved.

Hands-On: String Slicing

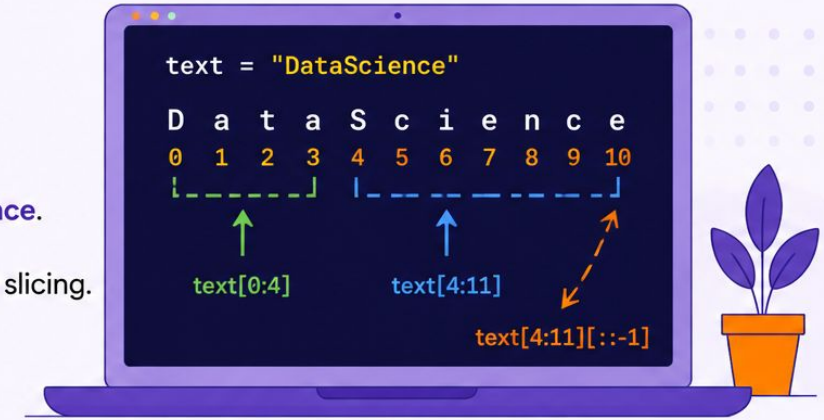
© Copyright Intellipaat. All rights reserved.

Hands-On: String Slicing

Problem Statement

Given `text = "DataScience"`,

- Use slicing to display **Data** and **Science**.
- Also reverse the string **Science** using slicing.



Expected Output

```
First Part: Data
Second Part: Science
Reversed String: ecneicS
```



© Copyright Intellipaat. All rights reserved.

String Operations

© Copyright Intellipaat. All rights reserved.

String Operations

Strings support various operations that help us **combine**, **repeat**, **check**, and **measure** text easily.

These operations make working with text simple and powerful.

In this section, you will learn:

1 Concatenation (Joining Strings)



Combine two or more strings using the `+` operator.

2 Repetition (Repeating Strings)

a a a

Repeat a string multiple times using the `*` operator.

3 Membership (in / not in)



Check if a substring exists in a string using `in` and `not in`.

4 Length of String (len())



Find the number of characters in a string using `len()`.

Key Takeaway

These basic operations form the foundation for many powerful string manipulations in **Python**.

© Copyright Intellipaat. All rights reserved.

Concatenation

We can join two or more strings together using the + operator.

Example

```
first  = "Hello"
second = "World"
full   = first + " " + second
print(full)
```

Output

```
Hello World
```

Explanation

- The + operator combines strings end to end.
- If we want a space between two words, we add " " (a space) between them.
- Strings are not changed by concatenation; a new string is created.

How It Works

Step 1: Join first and a space



Step 2: Then join with second



`final result → "Hello World"`

Key Takeaway

Use the + operator to join strings together.
You can also join more than two strings in the same way!

© Copyright Intellipaat. All rights reserved.

Repetition

We can repeat a string multiple times using the * operator.

Example

```
text = "Hi"
result = text * 3
print(result)
```

Output

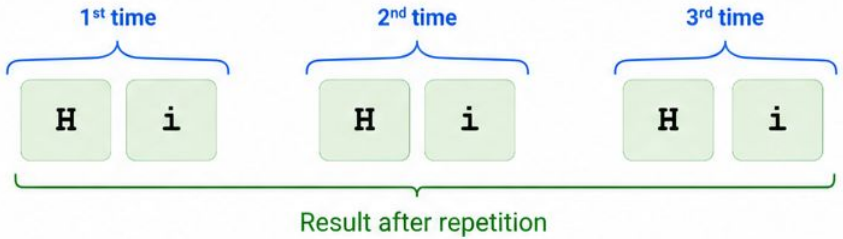
```
HiHiHi
```

Explanation

- The * operator repeats the string the specified number of times.
- The original string remains unchanged.
- The number can be any positive integer (0, 1, 2, 3, ...).

How It Works

If `text = "Hi"` and we do `text * 3`



`text * 3` → `"HiHiHi"`

If we multiply by 0, we get an empty string.
Example: `"Hi" * 0` → `""`

Key Takeaway

Use the * operator to repeat a string multiple times quickly and easily.

© Copyright Intellipaat. All rights reserved.

Length of String

We can find the number of characters in a string using the **len()** function.

Example

```
text = "Python"
length = len(text)
print(length)
```

Output

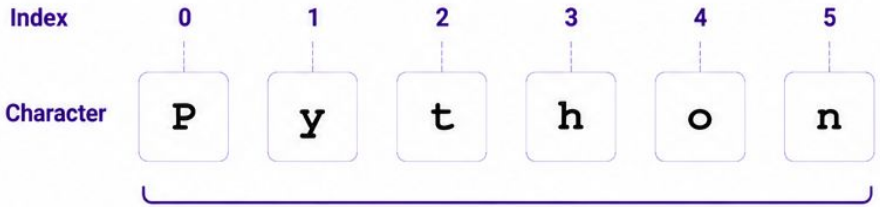
```
6
```

Explanation

- **len()** returns the total number of characters in the string.
- It counts letters, digits, spaces, and special characters as well.
- If the string is empty, **len()** returns **0**.

How It Works

```
text = "Python"
```



Total characters = 6 → `len("Python")` = 6

Empty string example:

`len("")` → 0

Key Takeaway

Use **len()** to quickly find how many characters are in a string.

© Copyright Intellipaat. All rights reserved.

Hands-On: String Operations

© Copyright Intellipaat. All rights reserved.

Hands-On: String Operations

Problem Statement

Create `first_name = "Rahul"` and `last_name = "Sharma"`.

- Join them to form a **full name**.
- Print a **separator** using repetition.
- Display the **length** of the full name.

```
first_name = "Rahul"
last_name = "Sharma"
full_name = first_name + " " + last_name
separator = "=" * 10
length = len(full_name)
```



Expected Output

```
Full Name: Rahul Sharma
Separator: ==========
Length: 13
```



© Copyright Intellipaat. All rights reserved.

String Methods

© Copyright Intellipaat. All rights reserved.

String Methods

Python provides many **built-in methods** that let us perform common operations on **strings** easily and efficiently.

String methods are functions that are called on a string using **dot notation**.

```
string_object.method_name()
```

In this section, you will learn some essential string methods:

1 Case Conversion

Aa

Change the case of characters in a string using **upper()**, **lower()**, and **title()**.

2 Whitespace Handling

Remove or manage extra spaces using **strip()**, **lstrip()**, **rstrip()**.

3 Splitting Strings

Split a string into a list of substrings using **split()**.

4 Replace Method

Replace a part of a string with another using **replace()**.

5 Search Methods

Find a substring or count occurrences using **find()** and **count()**.

6 And More!

Explore other useful methods that make text handling powerful and easy.

Key Takeaway

These methods help us manipulate and work with text in more powerful and convenient ways.

© Copyright Intellipaat. All rights reserved.

Case Conversion Methods

We can change the case (uppercase, lowercase, title case) of strings easily.

Example

```
text = "python programming"
print(text.upper())
print(text.lower())
print(text.title())
```

Output

```
PYTHON PROGRAMMING
python programming
Python Programming
```

Explanation

- upper() converts all characters to uppercase.
- lower() converts all characters to lowercase.
- title() converts the first character of each word to uppercase.

Methods

Method	Description	Example
`upper()`	Converts all characters to UPPERCASE.	`text.upper()` → `PYTHON PROGRAMMING`
`lower()`	Converts all characters to lowercase.	`text.lower()` → `python programming`
`title()`	Converts the first character of each word to uppercase.	`text.title()` → `Python Programming`

Note:
These methods **do not** change the original string. They return a **new** string.

```
text = "python"
new_text = text.upper()
# text is still "python"
```

Key Takeaway

Use `upper()`, `lower()`, and `title()` to quickly convert the case of strings for better text formatting.

© Copyright Intellipaat. All rights reserved.

Whitespace Handling Methods

We can remove or manage extra spaces from the **beginning** and/or **end** of strings.

Example

```
text = "     hello world     "
print(text.strip())
print(text.lstrip())
print(text.rstrip())
```

Output

```
hello world
hello world
     hello world
```

Explanation

- **strip()** removes spaces from both the beginning and the end.
- **lstrip()** removes spaces from the **beginning** (left side).
- **rstrip()** removes spaces from the end (right side).

Methods

Method	What it does	Example	Result
`strip()`	Removes spaces from **both** the beginning and the end.	`text.strip()` → `hello world`	`hello world`
`lstrip()`	Removes spaces from the **beginning** (left side).	`text.lstrip()` → `hello world`	`hello world`
`rstrip()`	Removes spaces from the **end** (right side).	`text.rstrip()` → `hello world`	`hello world`

By default, these methods remove spaces (" ", \t, \n). You can also specify other characters to remove.

```
text = "  xxxhello worldxxx  "
text.strip("x")   → "hello world"
text.strip("xyz") → "hello world"
```

Key Takeaway

Use `strip()`, `lstrip()`, and `rstrip()` to clean up unwanted spaces and make your text neat and consistent.

© Copyright Intellipaat. All rights reserved.

Splitting Strings

We can split a string into a list of substrings based on a delimiter using the **split()** method.

Example

```
text = "apple,banana,mango"
result = text.split(",")
print(result)
```

Output

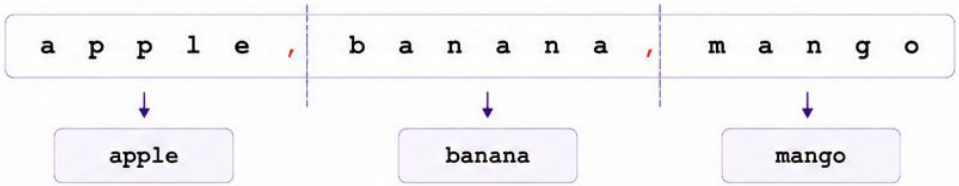
```
['apple', 'banana', 'mango']
```

Explanation

- The **split()** method divides the string at each occurrence of the given delimiter.
- It returns a **list** of substrings.
- If no delimiter is provided, it splits on any **whitespace** by default.

How It Works

```
text = "apple,banana,mango"
```




```
text.split(",") → ['apple', 'banana', 'mango']
```

More Examples

Code	Description	Output
`"a-b-c-d".split("-")`	Split by `'-'`	`['a', 'b', 'c', 'd']`
`"one two three".split()`	Split by whitespace (default)	`['one', 'two', 'three']`
`"python".split("x")`	Delimiter not found	`['python']`

Key Takeaway

Use **split()** to break a string into parts and get a **list** of substrings. It's very useful for processing text data.

© Copyright Intellipaat. All rights reserved.

Search Methods

We can search for substrings in a string and count their occurrences.

Example

```
text = "banana"
print(text.count("a"))
print(text.find("n"))
```

Output

```
3
2
```

Explanation

- `count()` returns the number of occurrences of a substring.
- `find()` returns the index of the first occurrence of a substring.
- If the substring is not found, `find()` returns `-1`.

count() Method

Returns the number of times a substring appears in the string.

Code	Output
`"banana".count("a")`	3
`"banana".count("na")`	2
`"hello".count("l")`	2
`"hello".count("z")`	0

find() Method

Returns the index of the first occurrence of a substring.

Returns `-1` if not found.

Code	Output
`"banana".find("n")`	2
`"banana".find("na")`	2
`"hello".find("l")`	2
`"hello".find("z")`	-1

Note:

- `count()` counts all occurrences.
- `find()` finds only the first occurrence.
- Both methods are case-sensitive.

```
text = "Python Programming"
text.count("P")      # 2
text.find("P")       # 0
```

Key Takeaway

Use `count()` to know how many times a substring appears and `find()` to know where it appears first.

© Copyright Intellipaat. All rights reserved.

More String Methods

Python provides many built-in methods to work with strings. Here are some commonly used ones.

Method	Description	Example	Output
startswith(prefix)	Checks if the string starts with the specified prefix.	`"hello world".startswith("hello")`	`True`
endswith(suffix)	Checks if the string ends with the specified suffix.	`"hello world".endswith("world")`	`True`
isalpha()	Returns True if all characters are alphabets.	`"Python".isalpha()`	`True`
isdigit()	Returns True if all characters are digits.	`"12345".isdigit()`	`True`
isalnum()	Returns True if all characters are alphanumeric (letters or digits).	`"abc123".isalnum()`	`True`
lower()	Converts all characters to lowercase.	`"PYTHON".lower()`	`python`
upper()	Converts all characters to uppercase.	`"python".upper()`	`PYTHON`
title()	Converts the first character of each word to uppercase.	`"hello world".title()`	`Hello World`
join(iterable)	Joins elements of an iterable with the string.	`",".join(["red", "green", "blue"])`	`red,green,blue`



Key Takeaway

String methods help us perform many useful operations easily, such as checking, converting, searching, and joining.

© Copyright Intellipaat. All rights reserved.

Hands-On: String Methods

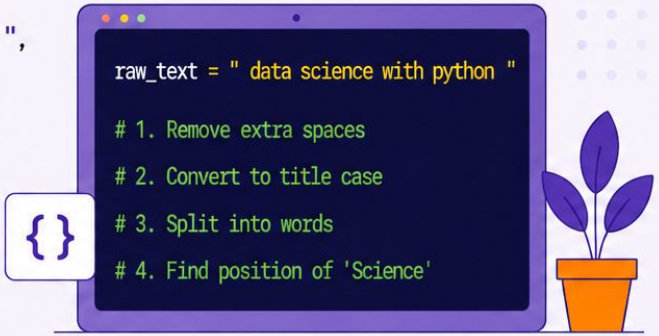
© Copyright Intellipaat. All rights reserved.

Hands-On: String Methods

Problem Statement

Given raw_text = " data science with python ",

- Remove the extra spaces.
- Convert the text to title case.
- Split it into words.
- Find the position of Science.



Expected Output

```
Cleaned Text: Data Science With Python
Words: ['Data', 'Science', 'With', 'Python']
Position of Science: 5
```



© Copyright Intellipaat. All rights reserved.

String Formatting

© Copyright Intellipaat. All rights reserved.

String Formatting

Formatting (or interpolation) allows us to insert variables and values inside strings to create clean, readable, and dynamic messages.

What is String Formatting?

String formatting is the process of combining strings with variables or values to produce a final string. It helps us build output messages in a clear and readable way.

Why Format Strings?

- Improves **readability** of the code and output.
- Makes it easier to insert variables **dynamically**.
- Reduces the need for manual **concatenation**.
- Essential for generating **user-friendly** messages, reports, and logs.

Example

Hard to Read & Maintain

```
name = "Manan"
age = 24
print("My name is " + name + ", I am " + str(age) + " years old.")
```

→

Clean, Readable & Maintainable

```
name = "Manan"
age = 24
print(f"My name is {name}, I am {age} years old.")
```

Key Takeaway

String formatting helps us create dynamic, readable, and professional output by embedding values inside strings.
In this section, we will explore different ways to format strings in Python.

© Copyright Intellipaat. All rights reserved.

Concatenation and Formatting

We can combine strings and variables in different ways.
Formatting helps us write cleaner, shorter, and more maintainable code.

Two Common Approaches

1. Concatenation (Using + Operator)

We can join strings and variables using the + operator.
It works, but can become lengthy with multiple values.

```
first_name = "Aisha"
subject = "Python"
score = 92
message = "Student " + first_name + " scored "+ str(score) + " in "
          + subject + "."

print(message)
```

Output

```
Student Aisha scored 92 in Python.
```

2. Formatting (Recommended Approach)

We use formatting methods to insert variables into strings.
It is cleaner, easier to read, and more efficient.

```
first_name = "Aisha"
subject = "Python"
score = 92
message = f"Student {first_name} scored {score} in {subject}."


print(message)
```

Output

```
Student Aisha scored 92 in Python.
```

Key Takeaway

Both methods produce the same result, but formatting is shorter, cleaner, and easier to maintain—especially when working with multiple variables or dynamic values.

© Copyright Intellipaat. All rights reserved.

format() Method

The `format()` method can be used with different data types like strings, numbers, and floating-point values.

How it Works

1. Use `{}` placeholders in the string.
2. Call the **format()** method on the string.
3. Pass the values in the same order as the placeholders.

Syntax

```
"string with {} placeholders".format(val1, val2, ...)
```

Example

```
name = "Riya"
marks = 95.5
passed = True

message = "Hello {}, you scored {} marks " \
"and pass status is {}.".format(name, marks, passed)
```

Output

```
Hello Riya, you scored 95.5 marks
and pass status is True.
```

Key Takeaway

The **format()** method works with different data types.
It helps create clear and readable strings by inserting values in the right places.

© Copyright Intellipaat. All rights reserved.

Hands-On: String Formatting

© Copyright Intellipaat. All rights reserved.

Hands-On: String Formatting

Problem Statement

Create variables for the:

- Name: `Arjun`
- Age: `22`
- Score: `88.75`

Use the `format()` method to display them in one sentence.

```
name    =  "Arjun"
age     =  22
score   =  88.75

message = "Hello {} , you are {} years"
  " old and scored {} marks."
output  = message.format(name, age, score)
print(output)
```



Expected Output

Output:

```
Hello Arjun, you are 22 years old and scored 88.75 marks.
```



© Copyright Intellipaat. All rights reserved.

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.